PCFFT IMPLEMENTATION NOTES

A PREPRINT

April 15, 2026

1 Computing the spreading grid

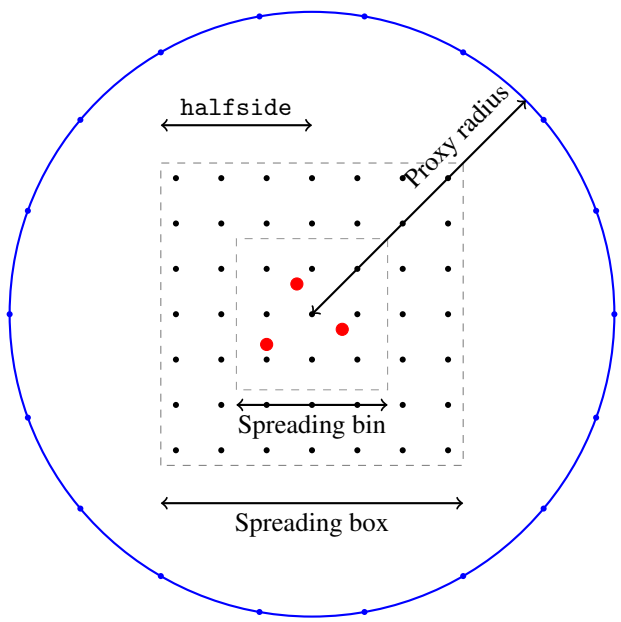

Figure 1: Schematic of the spreading geometry in 2D: sources (red), the regular discretization (black dots), and a proxy ring (blue).

1.1 Computing the spreading box size

The spreading box size is computed by `spread_halfside()`. This is meant to approximately control the number of source points in a particular spreading box.

1.2 Computing the regular grid spacing

This is performed in `dx_nproxy()`. We want to find parameters `dx` and `nproxy`. `dx` is the grid spacing of the regular discretization of the spreading box, which starts at $-\texttt{halfside} + \frac{\texttt{dx}}{2}$ and ends at $\texttt{halfside} - \frac{\texttt{dx}}{2}$. `nproxy` is the number of proxy points placed on a proxy ring (or sphere) outside the spreading box.

Notes on the geometry used:

- We put source points in a bin with sidelength $\texttt{c_bwidth} \times \texttt{halfside}$.

(a) $\texttt{nbinpts} = 3$, $\texttt{nspread} = 7$ **(b)** $\texttt{nbinpts} = 4$, $\texttt{nspread} = 8$

Figure 2: Schematic of the padding of the regular grid around the source points. The black lines form the bottom left corner of the bounding rectangle, the blue dashed line is the first spreading bin, and the orange dashed line is its associated spreading box. The regular grid points (black dots) start slightly below the bounding rectangle. There are $\texttt{pad} = \lceil(\texttt{nspread} - \texttt{nbinpts})/2\rceil$ extra grid points hanging below and to the left of the bounding rectangle to ensure proper padding for spreading.

- We place a proxy ring of radius $\sqrt{\texttt{d}} \times \texttt{halfside} \times \texttt{crad}$ where $\texttt{d}$ is the dimension (2 or 3) and $\texttt{crad}$ is a constant (default 2.0).
- If we consider breaking the spreading box into $\texttt{nspread}$ cells, the grid points are placed at the center of each cell, so $\texttt{dx} = \frac{2\times\texttt{halfside}}{\texttt{nspread}}$.

Notes on the algorithm used to compute $\texttt{dx}$ and $\texttt{nproxy}$:

- Generate random sources in the spreading bin with side length $\texttt{halfside}$. Generate target points on a ring/sphere of radius $1.1\times$ the proxy radius.
- Increase $\texttt{nspread}$ and $\texttt{nproxy}$ until the error tolerance is met.
- Decrease $\texttt{nspread}$ until the error tolerance is no longer met.
- Spreading bin is $\texttt{dx} \times \texttt{nbinpts} = \texttt{dx} \times \lfloor\texttt{nspread}/2\rfloor$.

1.3 Constructing the regular grid

This is performed in `get_grid()`, which calls `spread_halfside()` and `dx_nproxy()`. Here are the basic steps:

- Compute `halfside` using `spread_halfside()`.
- Compute `dx`, `nspread`, and `nproxy` using `dx_nproxy()`.
- Construct a bounding rectangle around the source points, which, along with the side length of the spreading bin, determines `ngrid`, the number of regular grid points in each dimension.
- Add a padding of grid points on each side so that the spreading bins fit properly into the corners of the bounding rectangle. $\texttt{pad} = \lceil(\texttt{nspread} - \texttt{nbinpts})/2\rceil$.
- Start the regular grid points a bit below the bottom corner of the bounding rectangle. This offset is $\texttt{offset} = \texttt{pad} * \texttt{dx} - \texttt{dx}/2$ See Figure 2 for an illustration of this padding.

2 Computing the spreading matrix

The spreading matrix $\boldsymbol{A}$ maps from weights on the source points to equivalent weights on the regular grid. This matrix is constructed in `get_spread()`.

[TODO: Describe the indexing of the spreading bins.]

[TODO: Describe the `SortInfo` object and data it contains.]

3 Computing the addsub matrix

3.1 Defining the neighborhood

For a spreading bin j, the neighboring bins are all such bins whose associated spreading boxes intersect the ball bounded by the proxy radius.

If we wanted to just include all bins which intersect the ball bounded by the proxy radius, we would use this cutoff in bin index space:

$$\texttt{idx_rad} = \lceil \frac{\texttt{proxy_rad}}{\texttt{dx} \times \texttt{nbinpts}} \rceil. \tag{1}$$

To make sure we are accounting for the overlap of the spreading boxes, we will inflate by the max size of the margin:

$$\texttt{idx_rad} = \lceil \frac{\texttt{proxy_rad} + \sqrt{d} \times \texttt{pad} \times \texttt{dx}}{\texttt{dx} \times \texttt{nbinpts}} \rceil. \tag{2}$$

3.2 Constructing the matrix

See the paper draft for the definition of the mathematical object we are trying to construct.

The algorithm takes these steps [TODO: Update this]

- Form a "spreading template", centered at bin i, which is a list of bins j such that $j \in \text{Near}(i)$.
- Use this "spreading template" to form a kernel matrix K_{bin2temp}, which has columns indexing the regular gridpoints of spreading bin i and rows indexing the regular gridpoints of the spreading template bins.
- For each bin i, return a list of each bin j where the proxy circle of i intersects with the proxy circle of j.
- Compute the exact near-field interactions $\sum_{j \in \text{Near}(x_i)} K(x_i - y_j)\mu_j$.
- Compute the contribution of the $\left(\boldsymbol{A}^{(\text{targ})}\right)^\top \tilde{\boldsymbol{K}} \boldsymbol{A}^{(\text{src})} \boldsymbol{\mu}$ term via indexing.

4 Derivatives

If we want to dipoles instead of point charges, we change the spreading matrix

$$\sum_{j \notin \text{Near}(x_i)} n_j \cdot \nabla_y K(x_i - y_j)\mu_j \approx \sum_k K(x_i - z_k) \boldsymbol{A}^d_{k,:} \boldsymbol{\mu},$$

We would then evaluate the bulk of the potential as

$$u = \boldsymbol{A}^T \tilde{\boldsymbol{K}} \boldsymbol{A}^d \boldsymbol{\mu}, \tag{3}$$

If we want the derivate of the potential due to point charges, we would instead evaluate

$$\partial_n u = (\boldsymbol{A}^d)^T \tilde{\boldsymbol{K}} \boldsymbol{A} \boldsymbol{\mu}, \tag{4}$$

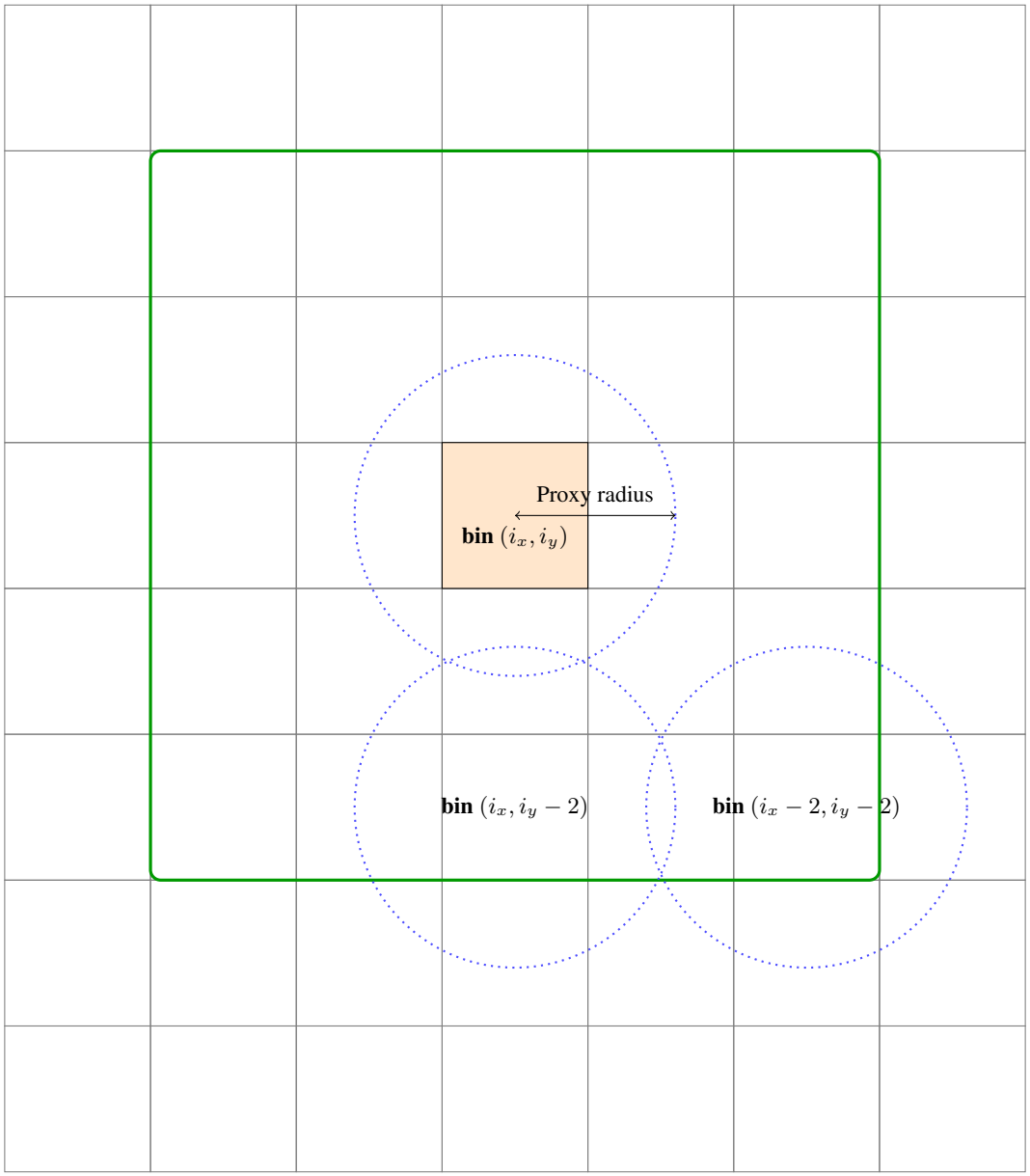

Figure 3: A schematic of the spreading template. The spreading template is always a square, found by taking the x and y bin indices which are within $2\frac{\texttt{proxy_rad}}{\texttt{bin_width}}$ of (i_x, i_y).